



## 第二章 同态加密

# 目 录

「01」

知识点一：基本概念

「02」

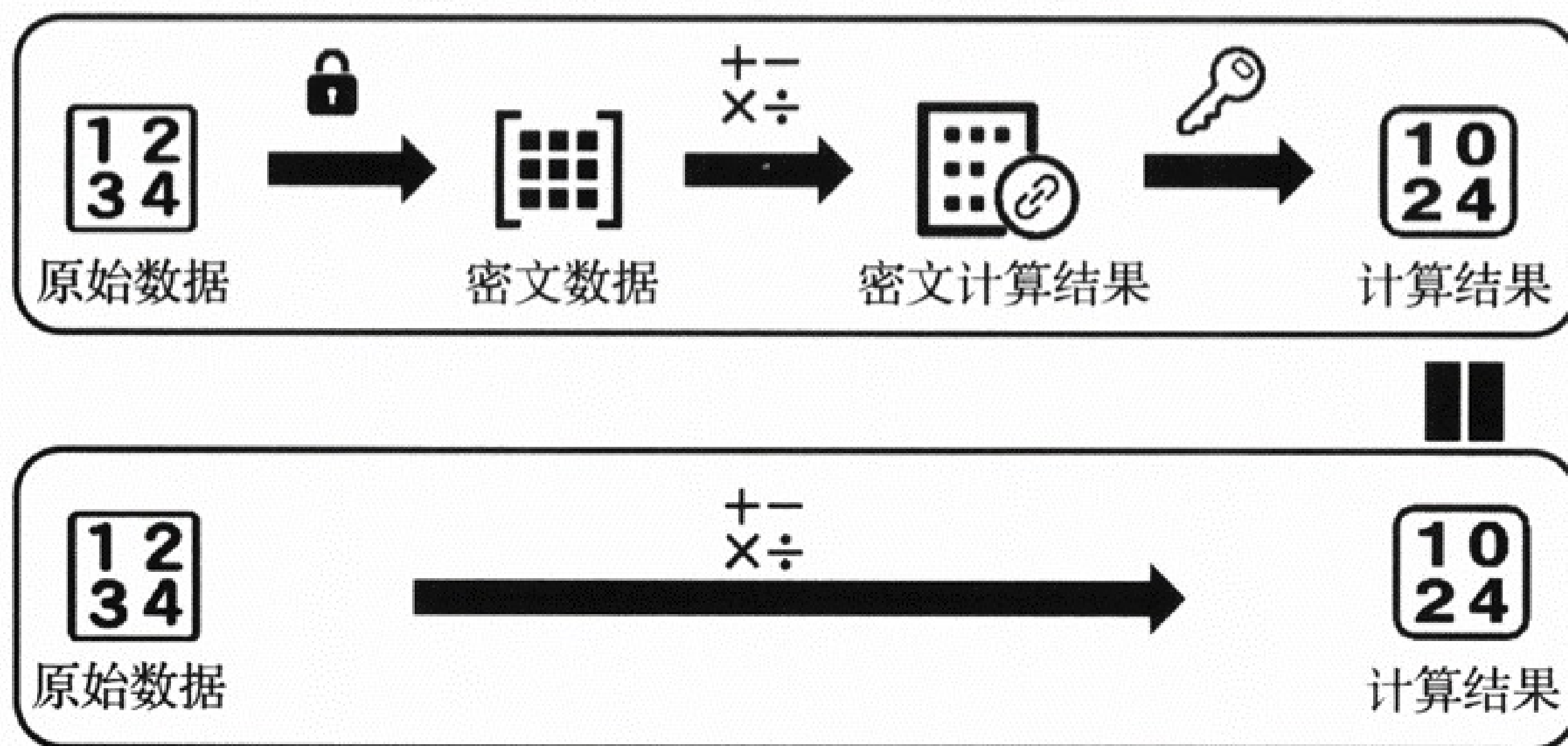
知识点二：半同态Paillier方案

「03」

知识点三：Paillier应用实践

# 1. 定义

- 同态加密(HE, homomorphic encryption)是一种加密算法，**它可以通过对密文进行运算得到加密结果，解密后与明文运算的结果一致。**
- 同态加密主要基于公钥密码体制构建，它允许将加密后的密文发给任意的第三方进行计算，并且在计算前不需要解密，可以在不需要密钥方参与的情况下，在密文上直接进行计算。





# 1. 定义

同态加密方案由KeyGen、Encrypt、Decrypt和Evaluate 4个函数构成：

- $\text{KeyGen}(\lambda) \rightarrow (\text{pk}, \text{sk})$  : 密钥生成函数；在给定加密参数 $\lambda$ 后，生成公钥/私钥对  $(\text{pk}, \text{sk})$  。
- $\text{Encrypt}(\text{pt}, \text{pk}) \rightarrow \text{ct}$  : 加密函数；使用给定公钥 $\text{pk}$ 将目标明文数据 $\text{pt}$ 加密为密文  $\text{ct}$ 。
- $\text{Decrypt}(\text{sk}, \text{ct}) \rightarrow \text{pt}$  : 解密函数；使用给定密钥 $\text{sk}$ 将目标密文数据 $\text{ct}$ 解密为明文 $\text{pt}$ 。
- $\text{Evaluate}(\text{pk}, F, \text{ct}_1, \text{ct}_2, \dots) \rightarrow (\text{ct}'_1, \text{ct}'_2, \dots)$  : **求值函数；**给定公钥 $\text{pk}$ 与准备在密文上进行的运算函数 $F$ ，求值函数将一系列的密文输入  $(\text{ct}_1, \text{ct}_2, \dots)$  转化为密文输出  $(\text{ct}'_1, \text{ct}'_2, \dots)$  。

# 1. 定义

- 求值函数是同态加密方案不同于传统加密方案的部分。它的参数 $F$ 支持的运算函数种类决定了该同态加密方案支持的同态运算操作。
- 同态加密方案应满足正确性、语义安全性和简短性：
  - [1] 正确性 (Correctness)：一个同态加密系统必须要是正确的。具体来说，也就是加密之后的密文可以被成功解密，并且求值函数输出的密文也可以成功解密回原文。
  - [2] 语义安全 (Semantic Security)：同态加密系统输出的密文必须要难以分辨。具体来说，如果有一个网络窃听者看到了所有的密文，那么这个窃听者并不能分辨出哪个密文是对应哪个原文的。
  - [3] 简短性 (Compactness)：同态加密的求值函数输出的密文的长度需要在一个可以控制的长度范围内，确保了同态加密系统的实用性。

## 2. 分类

根据同态加密算法所支持的同态操作种类和次数，可以将现有同态加密方案分为以下几种类型：

- **半同态加密(partial homomorphic encryption, 简称PHE)**仅支持单一类型的密文域同态运算(加或乘同态)；
- **类同态加密(somewhat homomorphic encryption, 简称SHE)**能够支持密文域有限次数的加法和乘法同态运算；
- **层级同态加密(Leveled Homomorphic Encryption, 简称LHE)**能同时支持多种同态操作(加或乘同态)，并可以在安全参数中定义能够执行的操作次数上限。一般允许的操作次数越大，该同态加密方案的密文空间开销及各类操作的时间复杂度就越大；
- **全同态加密(fully homomorphic encryption, 简称FHE)**能够实现任意次密文的加、乘同态运算。



有关同态加密说法错误的是

- A 计算方可以在同态加密算法输出的密文上做计算
- B 同态加密求值函数输出的密文可以被解密
- C 同态加密的语义安全性意味着相同的明文应该加密成不同的密文
- D 类同态加密能同时支持多种同态操作(加或乘同态)，并可以在安全参数中定义能够执行的操作次数上限



### 3. 发展历史

| 类型    |     | 算法         | 时间    | 说明                          |
|-------|-----|------------|-------|-----------------------------|
| 半同态加密 |     | RSA算法      | 1977年 | 非随机化加密，具有乘法同态性的原始算法面临选择明文攻击 |
|       |     | ELGamal算法  | 1985年 | 随机化加密，乘法同态                  |
|       |     | Paillier算法 | 1999年 | 加法同态，在联邦学习中广泛应用             |
| 类同态加密 |     | BGN方案      | 2005年 | 支持任意次加法和一次乘法操作的同态运算         |
| 全同态加密 | 第一代 | Gentry方案   | 2009年 | 自举操作，性能差                    |
|       | 第二代 | BGV方案      | 2012年 | 基于算术电路，基于模归约提升了自举性能         |
|       |     | BFV方案      | 2012年 | 基于算术电路，使用SIMD操作提升了自举性能      |
|       | 第三代 | GSW方案      | 2013年 | 支持任意布尔电路，基于近似特征向量           |
|       |     | FHEW方案     | 2015年 | 支持任意布尔电路，可实现快速比较            |
|       |     | TFHE方案     | 2016年 | 支持任意布尔电路，基于近似特征向量           |
|       | 第四代 | CKKS方案     | 2017年 | 可实现浮点数近似计算                  |



## 3. 发展历史

### (1) 半同态加密

➤ **乘法同态加密**：是指存在有效算法 $\otimes$ ，使得 $Enc(x) \otimes Enc(y) = Enc(x \times y)$

或者 $Dec(Enc(x) \otimes Enc(y)) = x \times y$ 成立，并且不泄漏 $x$ 和 $y$ 。

典型乘法同态加密算法是RSA算法和ElGamal算法。

以RSA算法为例，如果 $c_1 = m_1^e \bmod n$ ， $c_2 = m_2^e \bmod n$ ，那么

$$c_1 \times c_2 = m_1^e m_2^e \bmod n = (m_1 m_2)^e \bmod n \equiv Enc(m_1 \times m_2)。$$

### 3. 发展历史

#### (1) 半同态加密

➤ **加法同态加密**：是指存在有效算法 $\oplus$ ，使得 $Enc(x) \oplus Enc(y) = Enc(x + y)$

或者 $Dec(Enc(x) \oplus Enc(y)) = x + y$ 成立，并且不泄漏 $x$ 和 $y$ 。

典型加法同态加密算法是Paillier算法，详见知识点二。

加法和乘法同态是相对明文而言所执行的操作，而非密文上执行的运算形式

### 3. 发展历史

#### (2) 类同态加密

- 类同态加密方案能够同时支持加法和乘法的同态操作。
- 但由于它生成的密文随着操作次数的增加而逐渐增大，能够在密文上执行的同态操作次数是有上限的。
- 典型的类同态加密方案是Boneh、Goh和Nissim在2005年提出Boneh-Goh-Nissim（简称BGN）方案，它支持在密文大小不变的情况下进行任意次数的加法和一次乘法。该方案中的加法同态基于类似Paillier算法的思想，而一次乘法同态基于双线性映射的运算性质。虽然该方案是双同态的（同时支持加法同态和乘法同态），但只能进行一次乘法操作。



## 3. 发展历史

### (3) Gentry方案（第一代全同态加密方案）

- 2009年，Gentry提出了“**自举**”（bootstrapping）技术，可以**将满足条件的类同态加密方案改造成全同态加密方案**。其基本思想是在类同态加密算法的基础上引入自举方法来控制运算过程中的噪声增长（类同态加密算法操作次数过多会导致噪声过大而无法解密），这也是第一代全同态加密方案的主流模型。
- 为了避免多次运算使得噪声扩大，Gentry方案采用了计算一次就消除一次噪声的方法，而消除噪声的方法还是使用的同态运算。但是，由于解密过程本身的运算十分复杂，运算过程中也会产生大量噪声，因此，**Gentry方案性能极差，一次同态乘法可能需要30分钟**。
- 现在的第四代全同态加密解决方案要比Gentry提出的方案要好得多，性能大概提高了100万倍，并且已经开始制定相关的标准。

### 3. 发展历史

#### (4) BGV和BFV方案（第二代全同态加密方案）

- 第二代全同态加密方案主要包括 BGV和BFV，安全性基于格困难问题。
- 第二代方案主要是解决自举操作带来的昂贵操作，**通过引入层级同态加密等来提升性能**。此外，第二代全同态加密还**提出了单指令多数据SIMD方案通过批量处理来提高吞吐量，极大降低了均摊复杂度**。
- SIMD操作，简单来说，把密文切出上千个槽，把上千个明文放在这些密文槽中，这样，就可以并行处理各个槽中的数据了。
- **第二代全同态加密方案的性能已经提升了很多，每个明文比特的自举时间约为0.9毫秒，自举一个密文能在10秒左右完成，具有了一定的实用性。HElib和SEAL两个全同态加密开源库均支持BGV和BFV方案。**

### 3. 发展历史

#### (5) TFHE等方案（第三代全同态加密方案）

- GSW、FHEW和TFHE是第三代同态加密方案重要的代表作。与第二代FHE方案相比，**自举的性能得到大幅度提升，在常见的台式机平台上速度可以达到毫秒级别。**
- 同时，因为缺少第二代FHE的SIMD特性，FHEW只能处理若干比特（典型值为2~7）的加法和乘法操作，也就是说**同态乘法的性能较差。**



### 3. 发展历史

#### **(6) CKKS等方案（第四代全同态加密方案）**

- CKKS (Cheon-Kim-Kim-Song) 方案支持针对实数或复数的浮点数加法和乘法同态运算，但是得到的计算结果是近似值。因此，它适用于不需要精确结果的场景。
- **支持浮点数运算这一功能在实际中有非常重要的作用，如实现机器学习模型训练等。**
- 这个方案的性能也非常优异，大多数算法库都实现了CKKS。



能够同时支持多种同态操作(加或乘同态), 并可以在安全参数中定义能够执行的操作次数上限的同态加密是

- ☐ A 半同态加密
- ☐ B 类同态加密
- ☐ C 层级同态加密
- ☐ D 全同态加密

下列哪个同态加密方案可以支持浮点数运算

- ☐ A Paillier
- ☐ B Gentry
- ☐ C CKKS
- ☐ D GSW

# 目 录

「01」

知识点一：基本概念

「02」

知识点二：半同态Paillier方案

「03」

知识点三：Paillier应用实践



# 1. 卡迈克尔函数

**卡迈克尔函数**的定义为：设 $\gcd(a, n) = 1$ ，使得 $a^m \equiv 1 \pmod{n}$ 成立的最小正整数 $m$ ，将 $m$ 记作 $\lambda(n)$ ， $\gcd$ 是求最大公约数。

➤ 对于 $n=pq$ ， $p$ 和 $q$ 都是素数，则有 $\lambda(n) = \text{lcm}(p-1, q-1)$ ， $\text{lcm}$ 为求最小公倍数。

**欧拉函数**：对正整数 $n$ ，欧拉函数是小于 $n$ 的正整数中与 $n$ 互质的数的数目：

➤  $\phi(1) = 1$ 。

➤ 对于 $m > 1$ ， $\phi(m)$ 就是 $\{1, \dots, m-1\}$ 中与 $m$ 互素的数的个数，比如说 $p = \{\text{素数}\}$ ，则有 $\phi(p) = p - 1$ 。

➤ 对于 $n=pq$ ， $p$ 和 $q$ 都是素数，则有 $\phi(n) = (p-1)(q-1)$ 。

# 1. 卡迈克尔函数

| n            | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|--------------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| $\lambda(n)$ | 1 | 1 | 2 | 2 | 4 | 2 | 6 | 2 | 6 | 4  | 10 | 2  | 12 | 6  | 4  | 4  |
| $\phi(n)$    | 1 | 1 | 2 | 2 | 4 | 2 | 6 | 4 | 6 | 4  | 10 | 4  | 12 | 6  | 8  | 8  |

- 对于8的卡迈克尔函数是2，即 $\lambda(8) = 2$ ，即对于任意的a满足 $\gcd(a, 8) = 1$ ，有 $a^m \equiv 1 \pmod{8}$ ，也就是说 $1^2 \equiv 1 \pmod{8}$ ， $3^2 \equiv 1 \pmod{8}$ ， $5^2 \equiv 1 \pmod{8}$ ， $7^2 \equiv 1 \pmod{8}$ 。
- 而对于欧拉函数来说， $\phi(8) = 4$ ，因为欧拉函数是计算与8互素的数的数量，即1、3、5、7。
- 对于 $n=15$ ，因为 $n=3*5$ ，令 $p=3$ ， $q=5$ ， $\lambda(15) = \text{lcm}(2,4) = 4$ ， $\phi(15) = 2 * 4 = 8$ 。

# 1. 卡迈克尔函数

## 卡迈克尔函数的特性:

设 $n = pq$ , 其中 $p$ 和 $q$ 是大素数, 那么 $\phi(n) = (p - 1)(q - 1)$ ,  $\lambda(n) = \text{lcm}(p - 1, q - 1)$ ,  $\text{lcm}$ 为求最小公倍数。为便于描述, 用 $\lambda$ 表示 $\lambda(n)$ 。

我们有,  $|\mathbb{Z}_{n^2}^*| = \phi(n^2) = n\phi(n)$ ,  $g^\lambda = 1 + kn, k \in \mathbb{Z}_n^*$ 。

其中,  $\mathbb{Z}_n^*$ 是定义在 $n$ 上的整数域, 但不含0元素。

对于任意 $g \in \mathbb{Z}_{n^2}^*$ , 有如下性质:

$$\begin{cases} g^\lambda \equiv 1 \pmod{n} \\ g^{n\lambda} \equiv 1 \pmod{n^2} \end{cases}$$



# 1. 卡迈尔克函数

具体推导如下：

根据 $\lambda$ 的定义： $\lambda = k_1(p - 1) = k_2(q - 1)$

根据费马小定理( $g^{p-1} \equiv 1 \pmod{p}$ ):

$$g^\lambda = g^{k_1(p-1)} = (g^{p-1})^{k_1} \equiv 1 \pmod{p}, \text{ 同理 } g^\lambda \equiv 1 \pmod{q}$$

所以： $g^\lambda \equiv 1 \pmod{pq} = 1 \pmod{n}$

所以： $g^\lambda = 1 + kn, k \in \mathbb{Z}_n^*$

结合上式及二项式定理，可得

$$g^{n\lambda} \pmod{n^2} = (1 + kn)^n \pmod{n^2} \equiv (1 + kn \cdot n) \pmod{n^2} \equiv 1 \pmod{n^2}$$

因为，设 $1 + n \in \mathbb{Z}_{n^2}^*$ ，有

$$(1 + n)^2 \equiv 1 + 2n + n^2 \equiv (1 + 2n) \pmod{n^2}$$

$$(1 + n)^3 \equiv 1 + 3n + n^3 \equiv (1 + 3n) \pmod{n^2}$$

$$(1 + n)^v \equiv 1 + v * n + [n\text{的高次幂}] \equiv (1 + v * n) \pmod{n^2}$$

## 2. 判定复合剩余假设

**剩余类：**亦称同余类，指全体整数按照对一个正整数的同余关系而分成的类。对于一个整数 $m$ ，可以把所有整数分成 $m$ 类，每类对于 $m$ 都同余（模 $m$ 后余数相同）。每一类都叫做 $m$ 的一个剩余类。比如整数5，有5个剩余类，对0同余的有 $\{-5, 0, 5, \dots\}$ 。

**复合剩余类：**如果存在一个数 $x \in Z_{n^2}^*$ ，那么符合公式 $z = x^n \bmod n^2$ 的数 $z$ ，称为 $x$ 的模 $n^2$ 的 $n$ 阶剩余类。或者说，如果数 $z$ 被称为 $x$ 的模 $n^2$ 的 $n$ 阶剩余类，则存在一个数 $x \in Z_{n^2}^*$ ，使得 $z = x^n \bmod n^2$ 。

**判定复合剩余假设 (decisional composite residuosity assumption, DCRA)：**设 $n = pq$ ， $p$ 与 $q$ 为两个大素数，对于任意给定的整数 $z$ ，判断它是否是模 $n^2$ 的 $n$ 阶剩余类是一个难解问题。



### 3. 算法描述

#### (1) 密钥生成

- 随机选择两个质数 $p$ 和 $q$ ，尽可能地保证 $p$ 和 $q$ 的长度接近或相等（安全性高）；
- 计算  $n = pq$  和  $\lambda = lcm(p - 1, q - 1)$ ，其中  $lcm$  表示最小公倍数；
- 随机选择  $g \in Z_{n^2}^*$ ，考虑计算性能优化，通常会选择  $g = n + 1$ ；
- 计算  $\mu = \left( L(g^\lambda \bmod n^2) \right)^{-1} \bmod n$ ，其中  $L(x) = \frac{x-1}{n}$ ；
- 公钥为  $(n, g)$ ；
- 私钥为  $(\lambda, \mu)$ 。

### 3. 算法描述

#### (2) 加密算法

对于任意明文消息  $m \in Z_n$ , 任意选择一个随机数  $r \in Z_n^*$ , 计算得到密文  $c$ :

$$c = E(m) = g^m r^n \bmod n^2$$

注意：密文  $c$  要比明文  $m$  长度要长。

#### (3) 解密算法

对于密文  $c \in Z_{n^2}^*$ , 计算得到明文  $m$ :

$$m = D(c) = L(c^\lambda \bmod n^2) * \mu \bmod n$$

### 3. 算法描述

#### 正确性

$$D(c) = L(c^\lambda \bmod n^2) * \mu \bmod n$$

$$= L((g^m r^n)^\lambda \bmod n^2) * \mu \bmod n$$

$$\underline{r^{n\lambda} \equiv 1 \bmod n^2}$$

$$= L((g^\lambda)^m \bmod n^2) * \left(L(g^\lambda \bmod n^2)\right)^{-1} \bmod n$$

$$= L((1 + kn)^m \bmod n^2) * (L((1 + kn) \bmod n^2))^{-1} \bmod n$$

$$\underline{(1 + n)^v \equiv 1 + v * n + [n\text{的高次幂}] \equiv (1 + v * n) \bmod n^2}$$

$$= L((1 + mk * n) \bmod n^2) * (L(1 + kn) \bmod n^2)^{-1} \bmod n$$

$$\underline{L(x) = \frac{x-1}{n}}$$

$$= mk * k^{-1} \bmod n$$

$$= m$$



### 3. 算法描述

#### 加法同态性

对于任意明文 $m_1, m_2 \in Z_n$ 和任意 $r_1, r_2 \in Z_n^*$ , 对应密文 $c_1 = E(m_1), c_2 = E(m_2)$ , 满足:

$$c_1 \cdot c_2 = g^{m_1} \cdot r_1^n \cdot g^{m_2} \cdot r_2^n \bmod n^2 = g^{m_1+m_2} \cdot (r_1 \cdot r_2)^n \bmod n^2$$

解密后得到:

$$D(c_1 \cdot c_2) = D(g^{m_1+m_2} \cdot (r_1 \cdot r_2)^n \bmod n^2) = m_1 + m_2$$

即我们得到了:  $c_1 \cdot c_2 = m_1 + m_2$ , 也就是, 密文乘等于明文加。

**注意:** 这里定义的密文加法运算形式是乘法运算, 但是因为运算的结果是明文相加, 因此是加法同态。同态是相对明文而言所执行的操作, 而非密文上执行的运算形式。

### 3. 算法描述

#### 标量乘法同态性

标量乘法：对于任意常数 $a$ ，满足 $E(a \times m) = a \times E(m)$ 。

对于明文 $m_1 \in Z_n$ 及其密文 $c_1$ ，给定一个整数 $a \in Z_n$ ，满足：

$$D(c_1^a \bmod n^2) = D(g^{m_1 a} \cdot (r^a)^n \bmod n^2) = m_1 a ,$$

**注意：**这里定义的密文标量乘运算形式是指数运算 $c_1^a$ ，但是因为运算的结果解密是常数乘明文 $m_1 a$ ，因此是标量乘法。

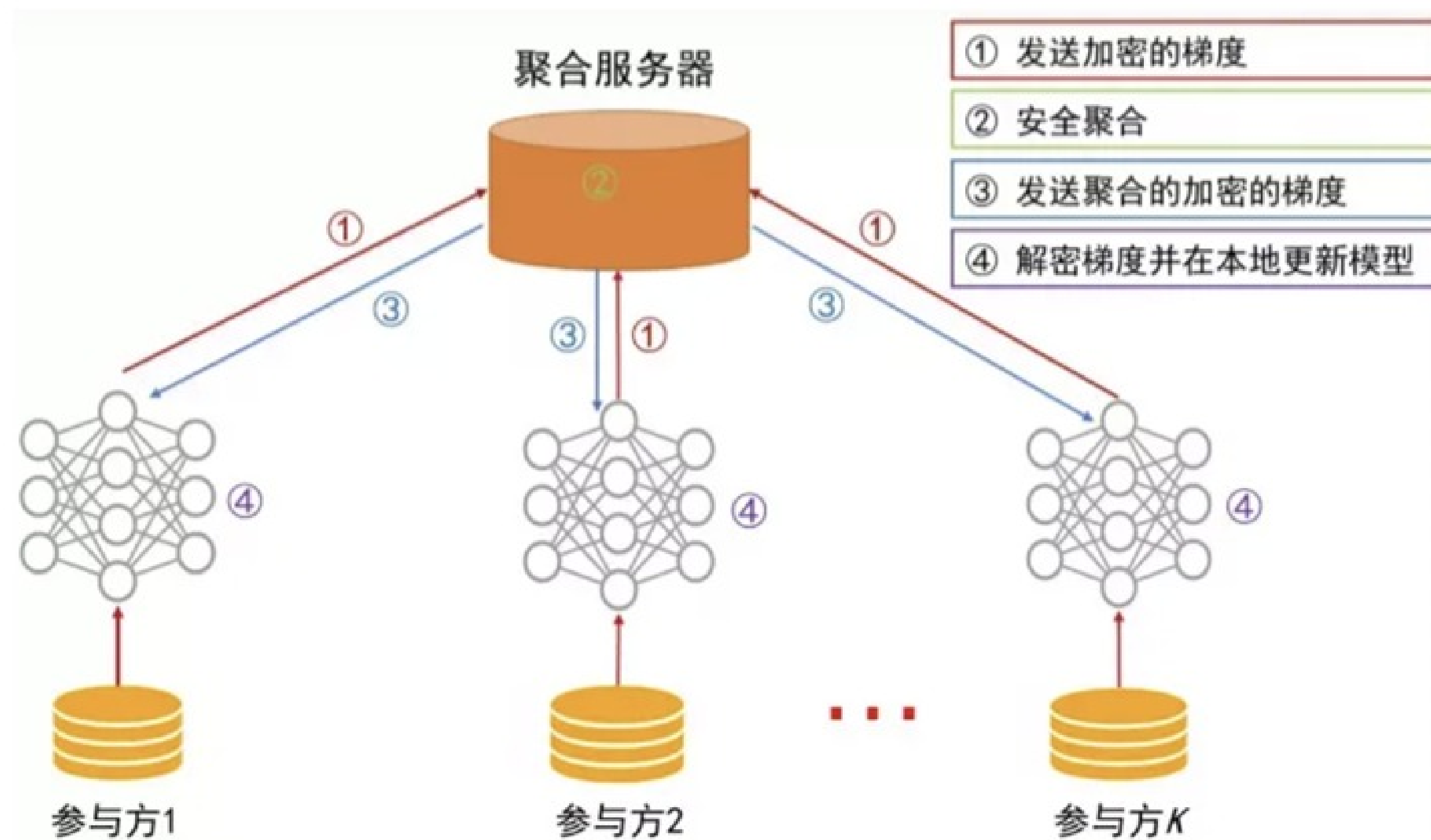
## 4. 典型应用

- 半同态加密虽然还不能同时支持加法和乘法运算，不能支持任意的计算，但是因为其与全同态相比，具有较高性能，因此，仍然具有极为广泛的应用场景，且在现实应用中起到了中重要的作用。
- 一类典型的应用体现在隐私保护的数据聚合上。由于加法同态加密可以在密文上直接执行加和操作，不泄露明文，在到多方协作的统计场景中，可完成安全的统计求和的功能。
- 在加密数据库SQL查询场景，在数据库不可信的情况下，可以通过部署协议和代理来保护请求者的查询隐私。其中，PHE可以用来完成安全数据求和、均值的查询。



## 4. 典型应用

**(1) 联邦学习：**在联邦学习中，不同参与方训练出的模型参数可由一个第三方进行统一聚合。使用加法PHE，可以在明文数据不出域、且不泄露参数的情况下，完成对模型参数的更新，此方法已应用在实际应用（如FATE）。



## 4. 典型应用

**(2) 隐私集合求和：** 在在线广告投放的场景中，广告主（如商家）在广告平台（如媒体）投放在线广告，并希望计算广告点击的转化收益。协议中的“隐私保护求和功能依赖于广告主将自己的交易数据用PHE加密发送给广告平台，使得广告平台在看不到原始数据的前提下，完成对交集中数据金额的聚合。该方案已被Google落地应用。





有关Paillier算法，说法错误的是

- ☐ A Paillier依赖的数学难题是判定复合剩余假设
- ☐ B Paillier可以用在密文上的求和平均值等运算
- ☐ C Paillier支持标量乘法，也就是密文与密文的乘积运算
- ☐ D Paillier属于半同态加密算法

# 目 录

「01」

知识点一：基本概念

「02」

知识点二：半同态Paillier方案

「03」

知识点三：Paillier应用实践

# 1. 实验环境安装

## (1) 安装python环境

□ 在Windows下安装python开发环境。到官方网站

<https://www.python.org/downloads/> 下载windows版本的python安装包。

下载后双击安装即可。

□ 提示：安装过程一定要勾选 “Add python.exe to PATH”，这样会使得安装后的python程序路径直接加入到系统的环境变量中，在控制台可以直接使用python命令。如果忘记勾选，则需要通过 “我的电脑” -> 右键 “属性” -> “高级系统设置” -> “环境变量” 的path中将安装的路径手动填入。



# 1. 实验环境安装

安装完毕，打开控制台，输入python命令，会显示：

```
命令提示符 - python
Microsoft Windows [版本 10.0.19044.2604]
(c) Microsoft Corporation。保留所有权利。

C:\Users\liuzh>python
Python 3.11.2 (tags/v3.11.2:878ead1, Feb 7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> _
```

代表已经安装成功，并且进入python运行环境。

**输入python程序：from phe import paillier**

该命令将导入phe库的paillier功能，第一次执行会提示 “ModuleNotFoundError: No module named 'phe'”。**因为，默认安装python后，并没有安装phe这个库。**

**输入python命令：exit()**

该命令可以退出当前python环境，切回控制台模式

# 1. 实验环境安装

## (2) 安装phe库

输入命令：pip install phe完成phe库的安装。

```
C:\Users\liuzh>pip install phe
Collecting phe
  Using cached phe-1.5.0-py2.py3-none-any.whl (53 kB)
Installing collected packages: phe
Successfully installed phe-1.5.0

[notice] A new release of pip available: 22.3.1 -> 23.0
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Pip是python的一个安装库的工具，可执行文件在python安装目录下可以找到。

## (3) 验证环境正确性

再次进入python环境，输入python代码：from phe import paillier  
发现不出现错误信息，说明环境安装成功。

# 1. 实验环境安装

## (4) 编写python程序并运行

可以有三种方式调试和编写python程序：

- 第一，在控制台运行python命令，逐行编写python程序并运行；
- 第二，用文本编辑器编写完整的程序并保存为x.py，通过控制台命令python x.py的方式完成整个程序的调用；
- 第三，通过IDLE这个python的集成开发环境完成开发和调试运行。通过开始菜单，找到IDLE并打开，选择File→new file可以新建一个文件，编辑程序并保存后，选择Run → Run Module运行，会看到运行的结果。



## 2. 简单示例

Python phe的库详见: <https://python-paillier.readthedocs.io/en/develop/usage.html#usage>。

这里给出一个集成的演示代码如下。

```
from phe import paillier # 开源库
import time # 做性能测试

##### 设置参数
print("默认私钥大小: ", paillier.DEFAULT_KEYSIZE)
#生成公私钥
public_key, private_key = paillier.generate_paillier_keypair()
# 测试需要加密的数据
message_list = [3.1415926, 100, -4.6e-12]
```

默认私钥大小: 3072

产生公私钥对

设置三个要加密的数据

## 2. 简单示例

##### 加密操作

```
time_start_enc = time.time()
```

```
encrypted_message_list = [public_key.encrypt(m) for m in message_list]
```

```
time_end_enc = time.time()
```

```
print("加密耗时s: ", time_end_enc - time_start_enc)
```

```
print("加密数据 (3.1415926) :", encrypted_message_list[0].ciphertext())
```

内嵌for循环  
使用public\_key.encrypt完成加密

##### 解密操作

```
time_start_dec = time.time()
```

```
decrypted_message_list = [private_key.decrypt(c) for c in encrypted_message_list]
```

```
time_end_dec = time.time()
```

```
print("解密耗时s: ", time_end_dec - time_start_dec)
```

```
print("原始数据 (3.1415926) :", decrypted_message_list[0])
```

内嵌for循环  
使用  
private\_key.decrypt完  
成解密



## 2. 简单示例

```
##### 测试加法和乘法同态
a,b,c = encrypted_message_list # a,b,c分别为对应密文
a_sum = a + 5 # 密文加明文, 已经重载了+运算符
a_sub = a - 3 # 密文加明文的相反数, 已经重载了-运算符
b_mul = b * 6 # 密文乘明文,数乘
c_div = c / -10.0 # 密文乘明文的倒数
```

重载了各类运算符

```
print("a+5 密文:",a.ciphertext()) # 密文纯文本形式
print("b*6=",private_key.decrypt(b_mul))
```

打印信息进行验证

##密文加密文

```
print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a+b))
#报错, 不支持a*b, 即两个密文直接相乘
#print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*b))
```

不支持密文乘,  
只支持标量乘

### 3. 隐私信息获取示例

实验2.2 基于Python的phe库完成隐私信息获取的功能：服务器端拥有多个数值，要求客户端能基于Paillier实现从服务器读取一个指定的数值并正确解密，但服务器不知道所读取的是哪一个。

首先，我们要基于Paillier协议进行设计。

对Paillier的标量乘的性质进行扩展，我们知道：

- 数值“0”的密文与任意数值的标量乘也是0；
- 数值“1”的密文与任意数值的标量乘将是数值本身。

如何利用这个特性来解决上述问题？

### 3. 隐私信息获取示例

**服务器端：** 产生数据列表  $\text{data\_list} = \{m_1, m_2, \dots, m_n\}$

**客户端：**

- 设置要选择的数据位置为  $\text{pos}$
- 生成选择向量  $\text{select\_list} = \{0, \dots, 1, \dots, 0\}$ ，其中，仅有  $\text{pos}$  的位置为 1
- 生成密文向量  $\text{enc\_list} = \{E(0), \dots, E(1), \dots, E(0)\}$
- 发送密文向量  $\text{enc\_list}$  给服务器

**服务器端：**

- 将数据与对应的向量相乘后累加得到密文

$$c = m_1 * \text{enc\_list}[1] + \dots + m_n * \text{enc\_list}[n]$$

- 返回密文  $c$  给客户端

**客户端：** 解密密文  $c$  得到想要的结果



```
from phe import paillier # 开源库
import random # 选择随机数
##### 设置参数
# 服务器端保存的数值
message_list = [100,200,300,400,500,600,700,800,900,1000]
length = len(message_list)
# 客户端生成公私钥，选择读取的位置
public_key, private_key = paillier.generate_paillier_keypair()
pos = random.randint(0,length-1)
##### 客户端生成密文选择向量
select_list=[]
enc_list=[]
for i in range(length):
    select_list.append( i == pos )
    enc_list.append( public_key.encrypt(select_list[i]) )
```

### 3. 隐私信息获取示例

```
##### 服务器端进行运算
```

```
c=0
```

```
for i in range(length):
```

```
    c = c + message_list[i] * enc_list[i]
```

```
print("产生密文: ",c.ciphertext())
```

```
##### 客户端进行解密
```

```
m=private_key.decrypt(c)
```

```
print("得到数值: ",m)
```

谢 谢

